



Cloud Foundry

Nitesh Pandey

By the end of this session, you should be able to understand:

❓ **Why monolithic applications had limitations ?**
and why microservices came into the picture

💡 **Making sense of the cloud:**
Cloud service models and where PaaS fits in

🔧 **What Cloud Foundry is ?**
The problems it solves for development and operations teams

🚀 **What it really means to push an application to Cloud Foundry ?**
Behind the scenes of `cf push`

📦 **How Cloud Foundry understands your application ?**
Buildpacks and their role in deployment

🕒 **Inside Cloud Foundry's architecture:**
The end-to-end journey of your source code — from your laptop → platform → running application

Monolith: Their advantages and disadvantages

A **monolithic application** is a single, self-contained application where all components — **User Interface, Server logic, and Database access** — are built, deployed, and run together as **one unit**.

Internally modular, but **released and upgraded as one unit** like early software distributed on CDs.

Advantages

- **Simple to develop and test** - One Codebase
- **Easy to deploy and operate** - One executable file
- **Simple initial scaling** by running multiple copies of the application behind a load balancer

Disadvantages

- As the application grows, it becomes **harder to understand slow to develop, modify, and test**.
- **Scalability** – You can't scale individual components.
- **Deployment:** Every change requires redeploying the entire application, increasing testing and deployment time across all environments
- **Difficult to adopt new technologies** - Any changes in the framework or language affects the entire application, making changes often expensive and time-consuming.



Microservices Architecture

Microservices architecture breaks an application into **small, independent services**, each responsible for a specific function and communicating with other services through **APIs**. Each service can be developed, deployed, scaled, and operated independently, making it a **cloud-native and distributed approach** to building applications.

Advantages

Easier to understand, develop, and maintain, as the codebase is divided into manageable services

Teams can **own individual services**, enabling faster development and parallel work

Loose coupling allows teams to adopt new technologies, frameworks, or languages independently

Independent deployments make changes, testing, and releases faster and safer

Fine-grained scaling, where only the required services are scaled instead of the entire application

Disadvantages

Troubleshooting and monitoring are more complex due to multiple services (requires good tooling)

Exponential infrastructure costs, Added organizational overhead, Debugging challenges .



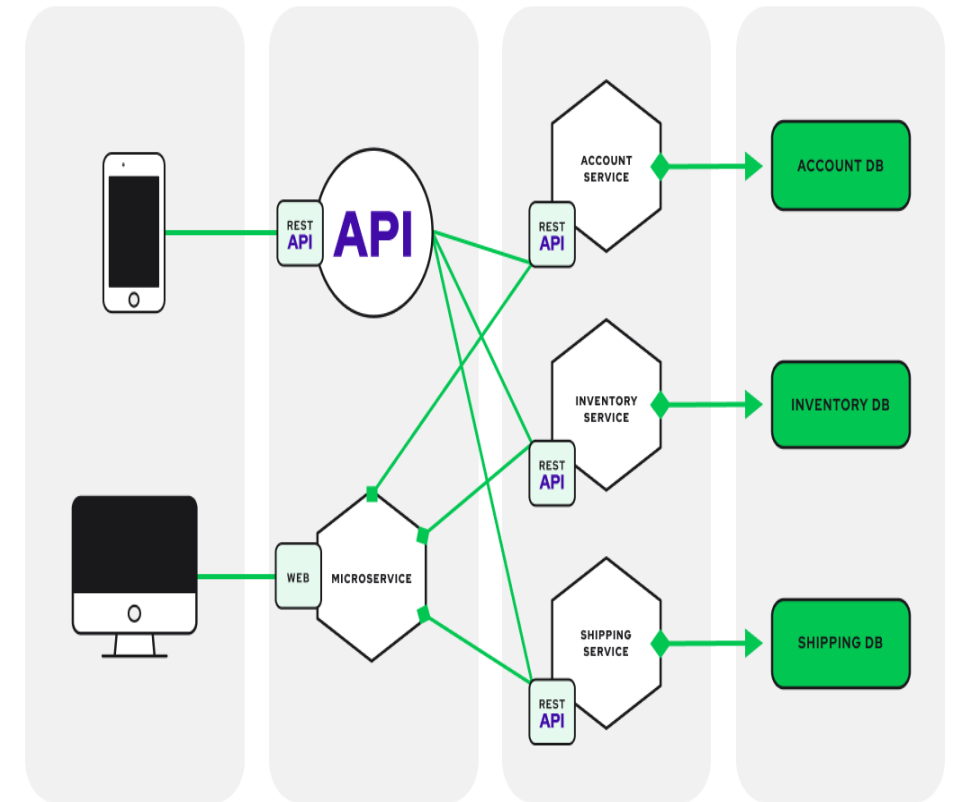
Summary: Modern Problems Need Modern Solutions

Modern web applications are built **for the cloud**, using a **cloud-native approach** where applications are split into small, distributed services that communicate over the internet.

While this improves agility and scalability, it also **increases deployment, scaling, and operational complexity**.

This is where Cloud Foundry comes in.

Cloud Foundry provides a **PaaS platform** that removes infrastructure complexity and allows teams to **focus on building, running, and managing microservices with confidence**.



Knowledge Check 1

Which statements describe a monolithic application? (Select all that apply)

- A) All components are deployed together**
- B) Each feature runs as an independent service**
- C) The application behaves as a single unit**
- D) Each component has its own database**

Knowledge Check 2

Which statements describe microservices correctly? (Select all that apply)

- A) Each service focuses on a single business capability**
- B) Services communicate using APIs**
- C) All services share the same database**
- D) Services must be deployed together**

☁ Making sense of the cloud:

Cloud service models and where PaaS fits in

Cloud models differ by how much responsibility you manage versus the provider.

- **SaaS — Order pizza and eat**
Use the software; no setup or maintenance
Examples: Gmail, Google Docs etc.
- **PaaS — Cook Pizza in a Fully Managed Kitchen**
Focus on code; platform manages runtime, scaling
Examples: Cloud Foundry
- **IaaS — Rent an Empty Kitchen**
Manage OS and applications; provider supplies servers
Examples: vSphere VMs, AWS EC2, Azure VMs



Where does Cloud Foundry Fits in ?

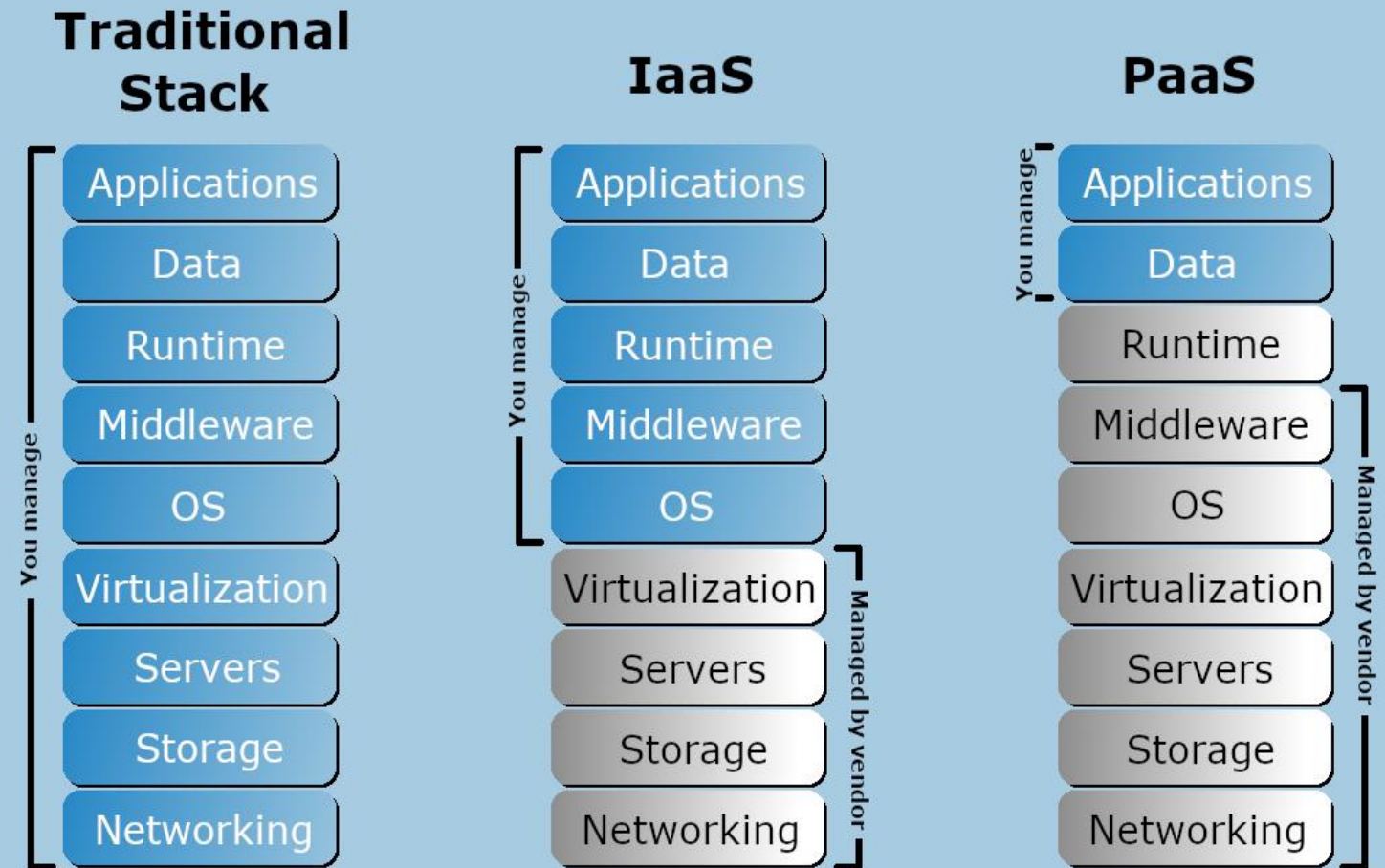
Cloud Foundry = PaaS

You bring:

- Application code
- Configuration

Cloud Foundry provides:

- Runtime & OS
- Buildpacks
- Containers
- Scaling, routing, health checks



Knowledge Check 1: Cloud Service Models

Which statements correctly describe cloud service models? (Select all that apply)

- A) SaaS requires users to manage servers and operating systems
- B) PaaS allows developers to focus on application code while the platform manages runtime and infrastructure
- C) IaaS provides virtualized computing resources like servers and networks
- D) SaaS gives full control over operating systems and middleware

Knowledge Check 2: Choosing the Right Model

Which cloud service model best fits each scenario?

- A) You want to use email without managing infrastructure
- B) You want to deploy custom applications without managing servers
- C) You want full control over OS and runtime on virtual machines
- D) You want to avoid writing any code

What exactly is Cloud Foundry ?

Cloud Foundry is an opinionated application platform (follows proven best practices) that brings efficiency, security, and speed to software development organizations. It makes it faster and easier to build, test, deploy, and scale applications by standardizing and automating operations (no manual infrastructure work), while providing flexibility in underlying infrastructure (VMs or Kubernetes), developer frameworks, and application services. This automation frees operators to focus on higher-value tasks, while developers get a secure self-service experience (push code and deploy safely on their own).

Enterprise distributions:

VMware Tanzu Application Service, SAP Cloud Foundry

Frees the development teams to focus on what they do best: **writing applications**.

By using Cloud Foundry throughout the development process, teams can **catch potential production problems** earlier and avoid nasty surprises in production.

Principle

Any infrastructure, Interoperability & Integration, Access to services

Container Platform, BOSH and Elastic Runtime

Containers are the technology that package application source code into **portable running processes** that behave the same from a developer's computer to production. **Cloud Foundry is a container-based platform** that uses this technology to run applications and manages how containers are built, started, scaled, and kept running, so developers do not need to worry about container details.

BOSH is the system that installs, runs, monitors, and repairs Cloud Foundry. It manages the entire lifecycle using a manifest file (a blueprint that describes all parts of the system and how they work together). BOSH installs an agent on each component so it can detect failures and automatically fix or restart them. Software is packaged as releases, which include code, OS dependencies, and configuration, ensuring the system is deployed the same way every time. This makes Cloud Foundry reliable, repeatable, and self-healing.

When we refer to Cloud Foundry as the **elastic runtime**, it is about how simple it is to grow or shrink to meet the needs of our customers. This allows operations teams to focus on improving application performance and availability instead of managing infrastructure, while developers can easily scale resources such as instances, memory, or disk using simple commands.

🚀 What it really means to push an application to Cloud Foundry ?

cf push — From Code to Live Application

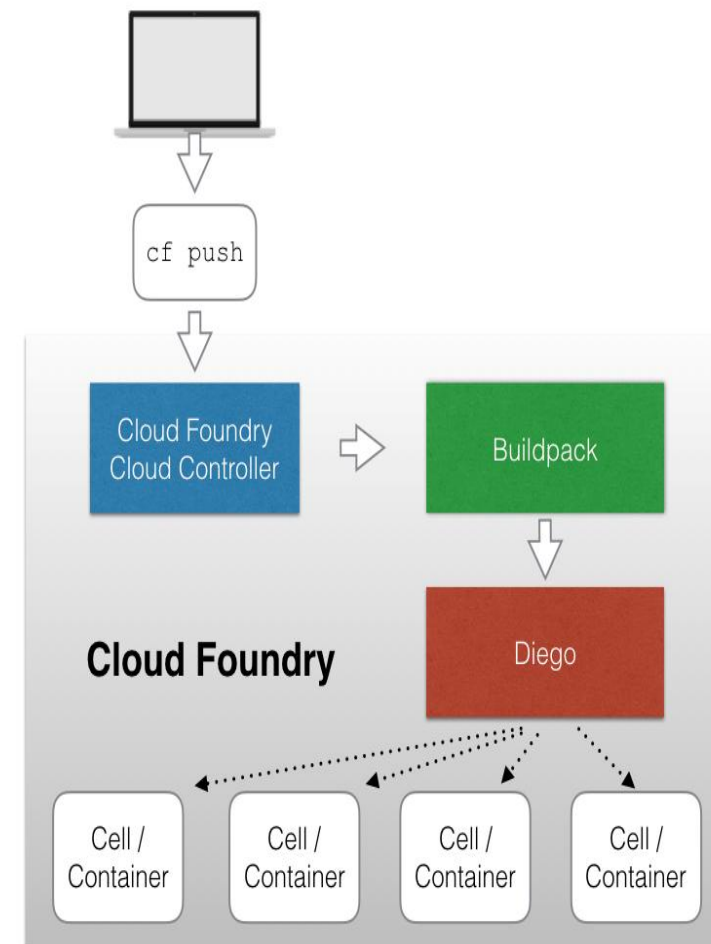
Cloud Foundry makes it easy to go from source code to a running cloud-native application with a single `cf push` command.

`cf push` handles the entire process—uploading the code, building the app, starting it, setting routes, and scaling resources—so developers can deploy, update, and manage applications directly from the command line with minimal effort.

The **CLI** gathers your application's files and uploads them to Cloud Foundry.

Within Cloud Foundry:

- The **Cloud Controller** finds the needed Buildpack.
 - The Buildpack builds an application image (**Droplet**).
 - **Diego** takes that **Droplet** and runs it in a Container (Diego Cell).
- Reports back success or any issues.



Journey of the Source Code

BUILD & DEPLOY

Developer

Laptop

|

| cf push

v

+-----+

| Cloud Controller |

+-----+

|

v

+-----+

| Blobstore |

<-- raw app bits

+-----+

|

v

+-----+

| Stager + |

| Buildpack |

<-- build runtime & deps

+-----+

|

v

+-----+

| Blobstore |

<-- droplet

+-----+

|

v

+-----+

| Diego Cell |

| (Garden) |

| App Running |

+-----+



RUNTIME TRAFFIC

User

|

DNS → Load Balancer → Gorouter → App Container → Response

PLATFORM SERVICES

Logs : App → Loggregator → cf logs / Splunk

Auth : UAA → Users, Tokens, Roles

Buildpacks

To run an application on Cloud Foundry, the required runtime and dependencies must be available. For example, a **Java application needs a JDK**, a **Node.js app needs Node.js**, and a **Python app needs Python and libraries**.

Developers **do not manually install or configure these in the manifest**.

When an application is pushed, **Cloud Foundry automatically detects the application type** and uses a **buildpack** to download and install everything required to build and run the app.

Eg.  **Node.js Application**

What the app needs:

Node.js runtime, npm packages (express, lodash, axios)

What Cloud Foundry does:

Detects package.json -> Downloads the correct Node.js version -> Runs npm install -> Prepares the app to start with npm start

Cloud Foundry packages:

Your code

Node.js runtime

All dependencies

Into ONE runnable unit called a: **Droplet**

 Think of a droplet as: “A ready-to-run app box”.

Cloud Foundry Architecture

At a high.. Very High level.. Cloud foundry can be understood as cluster of VMs

It is made of a control plane that manages applications and a runtime plane where applications actually run.

1. Component VMs create the underlying infrastructure for a deployment

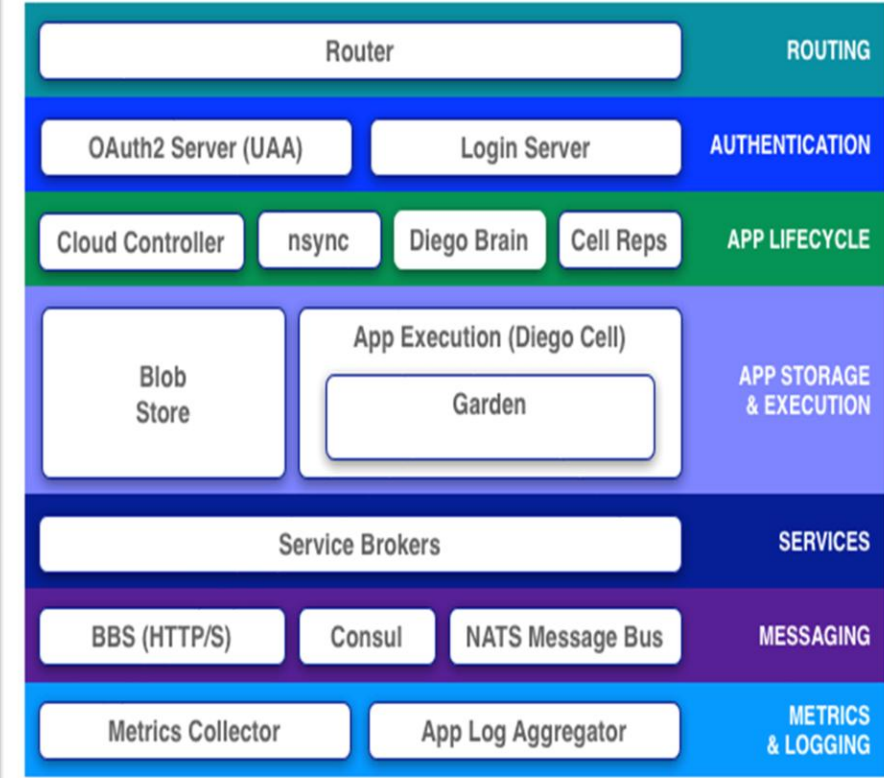
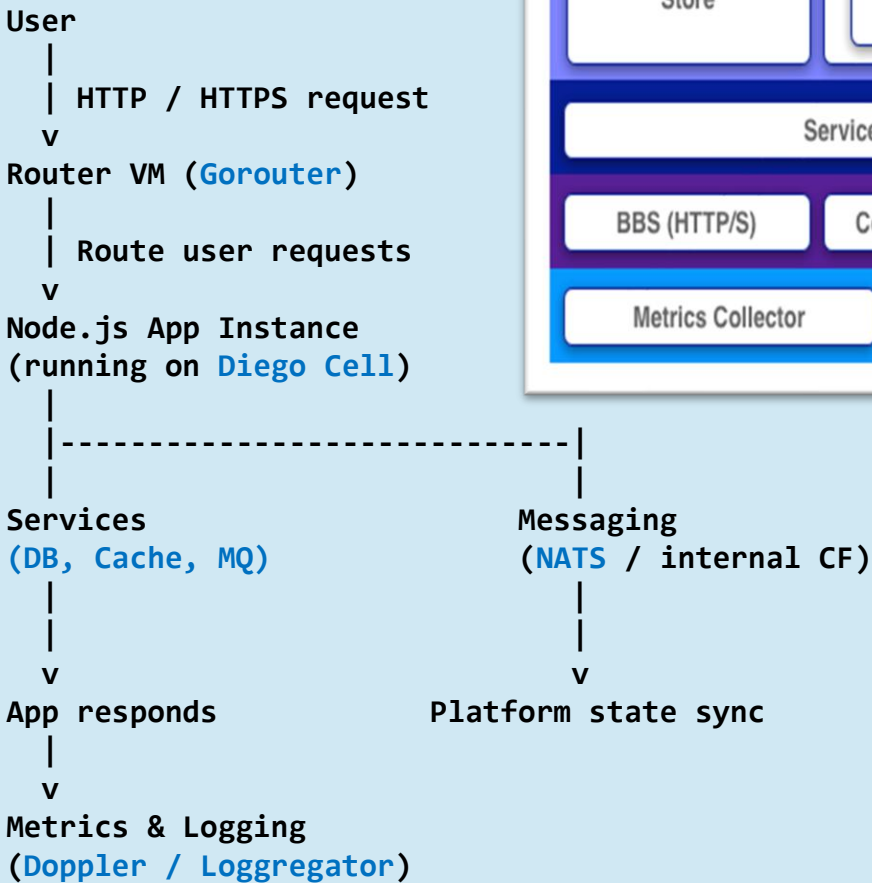
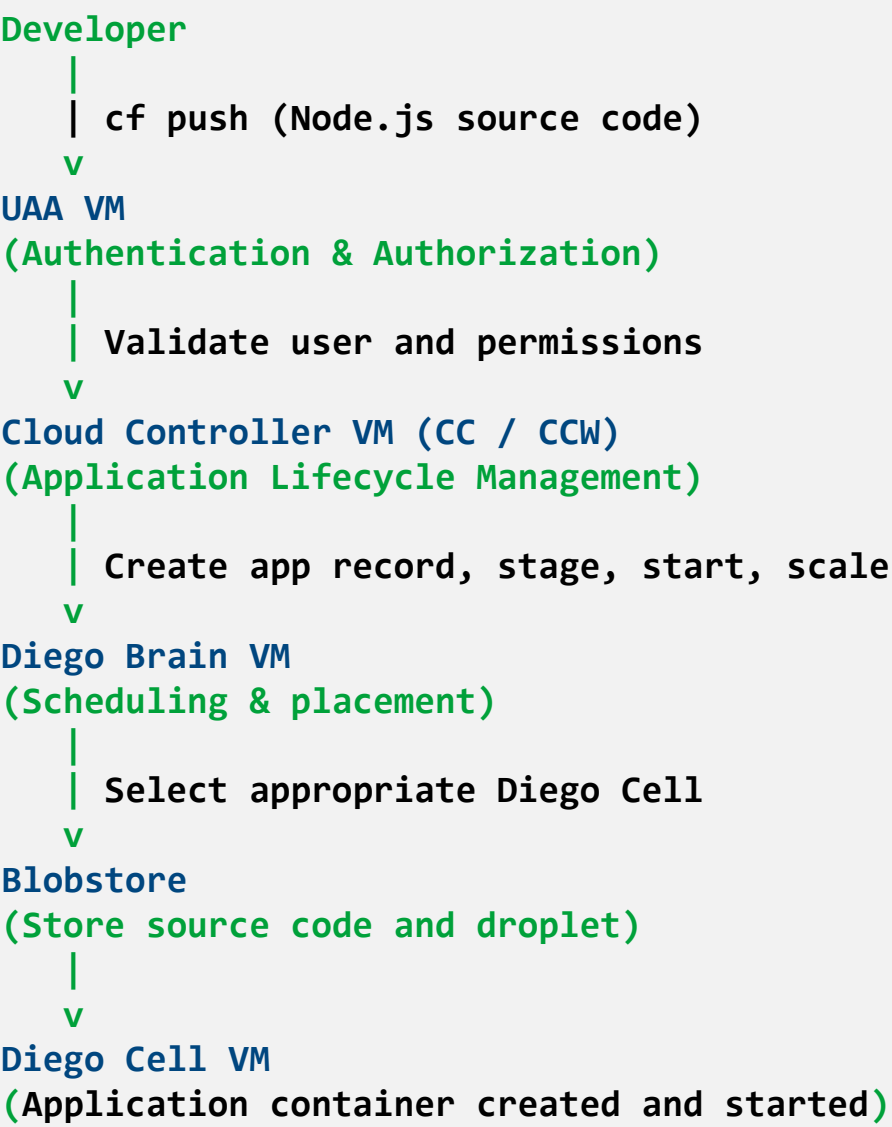
It does not run applications, but manages everything about them.

2. Host VMs provide a generic runtime environment for applications

It is where applications run and consume resources

Family	Role	VM Type [from a sample env]
Routing	Routes user traffic to apps	router
Authentication	User login and secrets	uaa, credhub
App Lifecycle	Push, stage, schedule apps	CC, CCW, diego_brain, clock_global, Diego_database
App Execution	Runs application containers	diego_cell
Services	Service binding for apps	(External eg. Redis, RMQ,)
Messaging	Internal component messaging	nats
Metrics & Logging	Logs and metrics collection	doppler, logg-traffic controller, log_cache
MySQL / mysql_proxy / mysql_monitor → persists Cloud Foundry platform data. backup_restore (BBR) → enables backup and restore of Cloud Foundry data stores		

The Journey of a Single Application Across the 7 Cloud Foundry Layers



Knowledge Check

1. What happens when a developer runs `cf push`?

- A. The developer provisions servers and containers
- B. Cloud Foundry builds, packages, and runs the app using platform components
- C. The app is deployed directly to a virtual machine
- D. User traffic is routed to the app

2. What is the purpose of Buildpacks?

- A. Install language dependent libraries
- B. Compile your application
- C. Provide Cloud Foundry with the directions on how to execute or run your application
- D. All of the above

3. Where are droplets stored in Cloud Foundry?

- A. A relational database
- B. A repository
- C. A blobstore
- D. An archived compressed file